

Bambu Farm Manager API 2.6

<This API document is compatible with farm manager server versions v02.00.03.00>

The Farm Server offers a range of fundamental API methods that empower you to develop custom functionalities and facilitate protocol-based control interactions with existing systems.

First, apply for API permission using your Bambu account. Next, utilize the Farm Manager client to activate the server and add printers. Finally, you can proceed with further development using the provided API methods.



- The request URL format includes the server's IP address and port, for example "https://192.168.1.100:8888/xxxxx", "https://127.0.0.1:8888/xxxxx".
- Both the https request and response bodies (except file uploading) are strings in JSON format
- The Common headers for http API:

Python

```
headers = {
    "Accept-Language": "en-US",
    "Accept": "[application/json, text/plain, */*]",
    "X-Bbl-App-Ver": "2.0.0-0",
    "Accept-Encoding": "[br, gzip, deflate]",
    "Authorization" : "token"
}
```

"Accept-Language" is set to "en-US" and currently, there is no support for multiple languages.

"X-Bbl-App-Ver" need to be fixed to be "2.0.0-0".

1. Client Authorization Certificates Update

certificate	Installation Location	Validity Period	Purpose
-------------	-----------------------	-----------------	---------

Client Certificate/Key	Third-party's application	12 months	• HTTPS mTLS handshake for Parter's Client and Farm Manager Server
------------------------	---------------------------	-----------	--

1.1 Apply for Client Certificate

The following API can be used to sign a new client certificate from bambu cloud. This allows third-party software to update the client certificate while keeping the private key inaccessible to Bambu Lab.

Note:

You can apply for only one certificate; please keep it secure. A replacement certificate can be requested only one month before the current one expires.

Request:

```

HTTP
Post https://api.bambulab.com/v1/iot-
service/api/farm/thirdparty/certs
headers = {
    "Accept-Language": "en-US",
    "Accept": "application/json",
    "X-Bbl-App-Ver": "2.0.0-0",
    "Accept-Encoding": "[br, gzip, deflate]",
    "x-bbl-client-name" : "BambuLocalServer",
    "x-bbl-client-type" : "local_server",
    "x-bbl-client-version" : "02.00.00.00" ,
    "user-agent" = "bbl-local-server/2.0",
    "x-bbl-device-id" = magic_id
}

body = {
    "pub_key": "-----BEGIN PUBLIC KEY-----\nXXXXXXXXXX\n-----
END PUBLIC KEY-----\n"
}

```

We recommend using RSA 2048 to generate the private key and public key. The "pub_key" should be provided as a PEM-formatted string. The "magic_id" will be provided by Bambu once the Bambu account has been granted API permission. This ID is associated with the specific Bambu account.

Response:

Plain Text

```
{
  "message": "",
  "code": 0,
  "cert": "-----BEGIN CERTIFICATE-----\nxxxxxxxx\n-----END
CERTIFICATE-----\n",
  "expire_at": "2026-05-21T04:06:28Z"
}
```

- "cert": The client's public certificate, provided as a PEM-formatted string.

1.2 Getting Server Status

This request could be used to detect whether the server is activated.

Request:

```
Python
GET /captain
```

Response:

```
JSON
{
  "name": "Bambu_997322",
  "power_limit": 0,
  "parallel_download": 10,
  "version": "11e50b built at 20241126142449",
  "server_id": "b4ff3124-5b15-4345-928e-841e0517d9d737fe",
  "activate_state": "Activated",
  "left_time": 0,
  "expire_time": "2024-11-27T07:47:21Z",
  "type": "release",
  "owner": "285123847",
  "release": "01.01.02.01",
  "serial_id": "186817375070072903762422807617144683296",
  "env": "",
  "user_improve_plan": "disable"
}
```

- "server_id": This is a unique identifier generated after the server is first run.
- "activate_state":
 - "Inactive": the server needs to be activated.
 - "Activated": the server can be used for printer control.
 - "Expired": The Certificate Sets has expired and needs to be reactivated.

- "left_time": The remaining days before the Server Certificate Sets expires.
- "expire_time": This field denotes the expiration time of the Server Certificate Sets. Prior to the expiration, the client must update the certificates as outlined in Section 1.4.
- "env": This refers to the region that will be automatically initialized during the activation process.

2. Local Account Token

The server uses a token-based access authorization for HTTP requests. We refer to this token as the "Local Account Token," and the token is valid for seven days. You can use the local account name and password to re-fetch the Local Account Token.

First, use the local account name to obtain a ticket. A challenge and a salt can be parsed from this ticket. Then, calculate a hash using the password, the received challenge, and the salt. Transmit the hash to the server. If the hash matches the server's calculation, a Local Account Token will be returned.

2.1 Fetch Local Account Ticket

Request:

```
HTTP
POST /login/local/ticket
{
  "user_name": "xxxxx"
}
```

Parameters:

"user_name": a local account name

Response:

```
JSON
{
  "message": "",
  "code": 0,
  "ticket_id": 2,

  "ticket": "eyJjaGFsbGVuZ2UiOiJXYkpSKyN5aiIsInNhbmhQI0iJvVWdsU3VkiUtaY1lzXzdNI0="
}
```

- "message": The error message.
- "code": int32. 0 for success, 1 for failure.

- "ticket_id": int64. A unique identifier for the ticket.
- "ticket": A string in base64 format, which contains challenge and salt. The ticket come from the operation: `base64({"challenge": "_4O!1#^>", "salt": "dUf-flr)*4_>UbnW"})`.

2.2 Fetch Local Account Token

Request:

```
HTTP
POST /login/local/tokens
{
  "user_name": "admin",

  "password": "e3b804aacc4b3a3fb4b2272f1d9ba9089352656db7bf57586c
2cab6e4249f6fd",
  "ticket_id": 2,
  "shared_client_key": "bbl_third_party"
}
```

Parameters:

- "user_name": The same account name used for the "fetch account ticket" operation.
- "ticket_id": The ID returned in response to the "fetch account ticket" operation.
- "password": this is password hash string. The challenge and salt come from the unbase64 operation for the "ticket" value and then calculated as `hex(sha256(hex(sha256(hex(sha256(password_plaintext))) + salt) + challenge))`.
- "shared_client_key": A fixed value set to "bbl_third_party".

Response:

```
JSON
{
  "message": "",
  "code": 0,
  "token": "eyJhbGciOiJSUz...",
  "shared_server_key": ""
}
```

- "message": The error message.
- "code": int32. 0 for success, 1 for failure.
- "token": A token in JWT format. This is the "Local Account Token" for this local

account.

- "shared_server_key": Not in use.

3. File Operation API

Each file and folder are associated with a unique ID. The file manager uses these unique IDs to manage files and folders, as well as to record their organizational relationships.

3.1 Add Folder

Request:

```
HTTP
Post /file/f3mffolder
{
  "name": "FolderName"
}
```

Response:

```
JSON
{
  "message": "",
  "code": 0,
  "folder": {
    "id": "1002",
    "name": "TestFolder",
    "create_time": "2025-01-26T05:54:37Z",
    "is_default": false
  }
}
```

- Https Status Code 200: Success.
- Https Status Code 400: Please check the error reason from the "message" field. For example, "folder name exist".
- "folder":
 - "id": The unique ID for this folder.
 - "name": The name of the folder.
 - "is_default": This field is not used. Only the default folder has this set to true. For all other folders, this value is always false.

3.2 Get Folder List

Request:

```
HTTP
GET /file/f3mffolder
```

Response:

```
JSON
{
  "total":3,
  "folders":[
    {
      "id":"1",
      "name":"",
      "create_time":"2024-11-26T07:45:56Z",
      "is_default":true
    },
    {
      "id":"2",
      "name":"cube_folder",
      "create_time":"2024-11-26T10:01:56Z",
      "is_default":false
    },
    {
      "id":"3",
      "name":"cube_folder/smal_cube",
      "create_time":"2024-11-26T10:02:09Z",
      "is_default":false
    }
  ]
}
```

3.3 Delete Folder

The folder can be deleted when it contains no files.

Request:

```
HTTP
DELETE file/{folder_id}/f3mffolder
```

Response:

```
JSON
{
  "message": "",
  "code": 0
}
```

- "code": 0 is success. 1 failed and the specific reason for the failure is provided in the message field.

3.4 Uploading 3MF file

Request:

```
HTTP
POST file/upload3mf
--boundary_string
Content-Disposition: form-data; name="folder_id"
1
--boundary_string
Content-Disposition: form-data; name="file"; filename="test.3mf"
Content-Type: application/octet-stream
```

Response:

```
JSON
{
  "message": "",
  "code": 0,
  "f3mf_info": {
    "id": "1",
    "create_time": "2024-11-26T10:02:20Z",
    "f3mf_name": "test.3mf",
    "folder_id": "1",
    "project_settings_config": {
      "print_compatible_printers": [
        "Bambu Lab A1 mini 0.4 nozzle"
      ],
      "curr_bed_type": "Textured PEI Plate",
      "filament_ids": [
        "GFA00",
        "GFA00",
        "GFA00"
      ]
    }
  },
}
```



```

    "slice_info_config":{
      "plates":[
        {
          "metadata":{
            "index":"1",
            "nozzle_diameters":"0.4",
            "prediction":"1033",
            "weight":"2.80",
            "support_used":"false"
          },
          "filaments":[
            {
              "id":"1",
              "type":"PLA",
              "color":"#FFFFFF",
              "used_m":"0.92",
              "used_g":"2.80"
            }
          ],
          "objects":[
            {
              "identify_id":"79",
              "name":"Cube",
              "skipped":"false"
            }
          ]
        }
      ],
      "model_setting_config":{
        "plate":[
          {
            "metadata":{
              "plater_id":"1",

```

"thumbnail_file":"3mf\\f3mf_4\\Metadata\\plate_1.png",

"top_file":"3mf\\f3mf_4\\Metadata\\top_1.png",

"pick_file":"3mf\\f3mf_4\\Metadata\\pick_1.png"

```

        }
      ]
    },

```

```

        "size":42290,
        "md5":"68f3d0cb01caba277267e2a37c20e9cf",
        "gcode_info":{
            "bed_target_temper":65
        }
    }
}

```

- "code": 0 is success. 1 failed and the specific reason for the failure is provided in the message field.
- "id": The unique identifier for this gcode.3mf file. This id will be used to create a task.

3.5 Get 3MF Files List

Request:

```

HTTP
GET /file/f3mf?folder_id={folder_id}

```

Response:

```

JSON
{
    "total": 1,
    "f3mf_info": [
        {
            "id": "1020",
            "create_time": "2025-01 26T10:06:54Z",
            "f3mf_name": "test.3mf",
            "folder_id": "1021",
            "project_settings_config": {
                "print_compatible_printers": [
                    "Bambu Lab A1 mini 0.4 nozzle"
                ],
                "curr_bed_type": "Textured PEI Plate",
                "filament_ids": [
                    "GFA00",
                    "GFA00",
                    "GFA00"
                ]
            },
            "slice_info_config": {
                "plates": [
                    {

```

```

        "metadata": {
            "index": "1",
            "nozzle_diameters": "0.4",
            "prediction": "1033",
            "weight": "2.80",
            "support_used": "false"
        },
        "filaments": [
            {
                "id": "1",
                "type": "PLA",
                "color": "#FFFFFF",
                "used_m": "0.92",
                "used_g": "2.80"
            }
        ],
        "objects": [
            {
                "identify_id": "79",
                "name": "Cube",
                "skipped": "false"
            }
        ]
    },
    "model_setting_config": {
        "plate": [
            {
                "metadata": {
                    "plater_id": "1",
                    "thumbnail_file":
"3mf\\f3mf_1023\\Metadata\\plate_1.png",
                    "top_file": "3mf\\f3mf_1023\\Metadata\\top_1.png",
                    "pick_file": "3mf\\f3mf_1023\\Metadata\\pick_1.png"
                }
            }
        ]
    },
    "size": 42290,
    "md5": "68f3d0cb01caba277267e2a37c20e9cf",
    "gcode_info": {
        "bed_target_temper": 65
    }
}

```

```
}  
]  
}
```

3.6 Delete 3MF File

Request:

```
HTTP  
DELETE /file/f3mf/{file_id}
```

Response:

```
JSON  
{  
  "message": "",  
  "code": 0  
}
```

3.7 Delete 3MF Files for A Folde

Request:

```
HTTP  
DELETE /file/f3mf/{file_id}
```

Response:

```
JSON  
{  
  "message": "",  
  "code": 0  
}
```

4. Printer Status & Control

4.1 Fetch Status for All Printers

Request:

```
HTTP  
GET /devices2
```

Response: This is an array of all the printers that are connected to the server.

```
JSON
{
  "devices":[
    {
      "dev_id":"01S09A2C0500292",
      "dev_model":"C11",
      "dev_name": "",
      "dev_ip":"192.168.3.126",
      "name":"P292",
      "report_status":{
        "nozzle_temper":25.375,
        "nozzle_target_temper":0,
        "bed_temper":23.3125,
        "bed_target_temper":0,
        "chamber_temper":5,
        "mc_print_stage":"1",
        "heatbreak_fan_speed":"0",
        "cooling_fan_speed":"0",
        "big_fan1_speed":"0",
        "big_fan2_speed":"0",
        "mc_percent":0,
        "mc_remaining_time":0,
        "ams_status":0,
        "ams_rfid_status":6,
        "hw_switch_state":1,
        "spd_mag":100,
        "spd_lvl":2,
        "print_error":83935248,
        "wifi_signal":"-60dBm",
        "gcode_state":"IDLE",
        "gcode_file_prepare_percent":"0",
        "project_id":"0",
        "profile_id":"0",
        "task_id":"0",
        "subtask_id":"0",
        "subtask_name": "",
        "gcode_file": "",
        "layer_num":0,
        "total_layer_num":0,
        "s_obj":[

        ],
        "hms":[
```

```
{
  "attr":50336256,
  "code":131073,
  "action":0,
  "timestamp":1732612487
}
],
"ams":{
  "ams_exist_bits":"1",
  "tray_exist_bits":"f",
  "tray_is_bbl_bits":"f",
  "ams":[
    {
      "id": "0",
      "humidity": "3",
      "temp": "0.0",
      "tray": [
        {
          "id": "0",
          "tag_uid": "0000000000000000",
          "tray_info_idx": "GFL00",
          "tray_type": "PLA",
          "tray_sub_brands": "",
          "tray_color": "F98C36FF"
        },
        {
          "id": "1",
          "tag_uid": "0000000000000000",
          "tray_info_idx": "GFA01",
          "tray_type": "PLA",
          "tray_sub_brands": "",
          "tray_color": "FFFFFFF"
        },
        {
          "id": "2",
          "tag_uid": "0000000000000000",
          "tray_info_idx": "GFA01",
          "tray_type": "PLA",
          "tray_sub_brands": "",
          "tray_color": "0ACC38FF"
        },
        {
          "id": "3",
          "tag_uid": "0000000000000000",
```

```

        "tray_info_idx": "GFA01",
        "tray_type": "PLA",
        "tray_sub_brands": "",
        "tray_color": "D4B1DDFF"
    }
    ]
}
]
},
"nozzle_diameter": "0.4",
"nozzle_type": "stainless_steel",
"stg": [

],
"stg_cur": 0,
"home_flag": 140723480,
"vt_tray": {
    "id": "255",
    "tray_info_idx": "GFA00",
    "tray_type": "PLA",
    "tray_sub_brands": "",
    "tray_color": "F98C36FF"
},
"vir_slot": [
    {
        "id": "255",
        "cols": [
            "F98C36FF"
        ],
        "cali_idx": -1,
        "ctype": 0,
        "nozzle_temp_max": "290",
        "nozzle_temp_min": "260",
        "tag_uid": "0000000000000000",
        "tray_color": "F98C36FF",
        "tray_id_name": "",
        "tray_info_idx": "GFA00",
        "tray_type": "PLA",
        "tray_sub_brands": ""
    }
],
"ip_cam": {
    "ipcam_dev": "1",
    "ipcam_record": "disable",

```

```

        "timelapse":"disable",
        "resolution":"","
        "mode_bits":3
    }
},
"tags":[
    {
        "tag_id":0,
        "tag_name":"G2",
        "color":"#e0e0e0"
    }
],
"online":true,
"running_task":null,
"optional_task":null,
"optional_task_cnt":0,
"sched_info":{
    "task_id":"","
    "sched_state":"no_task"
},
"liveview_url":"","
"liveview_time":"","
"upgrade_info":{
    "current_ver":"01.06.20.20",
    "next_ver":"","
    "previous_ver":"","
    "upgrade_state":"IDLE",
    "printer_status":"IDLE",
    "last_progress":"","
    "last_message":""
},
"print_cmd_send_retry_cnt":0,
"latest_print_cmd_send_fail_reason":"","
"sched_limited":"","
"is_downloading":false,
"is_heating":false
}
],
[...]
```

- "dev_id": The serial number of this printer.
- "dev_model": String, The model of the printer. "C11"(P1P), "C12"(P1S), "BL-P001"(X1C), N1(A1mini), N2S(A1) .

- "report_status": The status information reported by the printer.
 - "nozzle_temper|bed_temper": Double, current temperature.
 - "nozzle_target_temper|bed_target_temper": Int32, the target temperature.
 - "heatbreak_fan_speed": The hot end fan speed.
 - "cooling_fan_speed": The part cooling fan speed.
 - "big_fan1_speed": Auxiliary part cooling fan speed.
 - "big_fan2_speed": Chamber temperature regulator fan speed.
 - "mc_percent": Int32, the percentage of the printing process.
 - "mc_remaining_time": Int32, remaining time for printing in seconds.
 - "print_error": Int32, convert it to Hex value and find the cause from wiki. For example, 83935248 to 0500-C010, and then search the reason from "<https://wiki.bambulab.com/en/hms/error-code#:~:text=Both%20the%20HMS%20code%20and,page%20of%20the%20printer%20screen>".
 - "gcode_state": "PREPARE", "RUNNING", "PAUSE", "FINISH", "FAILED"
 - "gcode_file": The name of the gcode.3mf is currently being printed.
 - "gcode_file_prepare_percent": This represents the percentage completion during the preparation phase for downloading and extracting the G-code file on printer side.
 - "hms": Health Management System code for troubleshooting. "attr":50336256 convert to hex 03001200. "code":131073 convert to hex 00020001. The whole hms is 0300-1200-0002-0001 and then search the reason from "<https://wiki.bambulab.com/en/hms/home>".
 - vt_tray: This is filament information for the external spool and is valid for the P1/A1 series.
 - vir_slot: This is filament information for the external spool and is valid for the X1 series.
 - "ams":
 - "ams_exist_bits": bit0-bit3, indicating whether AMS is connected to the printer.
 - "tray_exist_bits": bit0-bit15, corresponding to slots A1~D16.
 - "ams": This is an array that provides detailed information about the filaments in the AMS slots.
- "tags": This is an array of tags assigned to this printer.

- "online": Indicates whether the printer is online.
- is_downloading: Indicates whether the printer is downloading a sliced file from the server.
- is_heating: Indicates whether the printer is heating the bed.

4.2 Fetch Status for Single Printer

Request:

```
Python
GET /device/printer_sn
```

Response: It will return information for only one device. Please refer to the response description for 'Fetch Status for All Printers' for details explanation.

4.3 Fetch Camera Picture

The printer will periodically upload camera images(jpeg) to the server at a low frequency to reduce network load. These images can be accessed via the provided API. You can compare the "liveview_time" field, such as "2024-11-27T02:40:36Z", to determine whether a new image has been uploaded and then fetch the latest image.

Request:

```
Python
Get /file?asset_path=liveview/{dev_id}/liveview.jpeg&x-bbl-sec-ver=1
```

- "dev_id": The serial number of the printer.

Example URL:

```
/file?asset_path=liveview/01P09A341000095/liveview.jpeg&x-bbl-sec-ver=1
```

Response:

- HTTP Status Code 200: Indicates success. The response body contains JPEG image data.

4.4 Stop/Pause/Resume/Bed_clean

This operational command will be converted into a printer-specific instruction and dispatched to the printer. Importantly, the server does not verify whether the printer is in the appropriate state to execute the command. Therefore, prior to initiating the operation, it is essential to perform a status check to ensure that the command is applicable to the current printing state("gcode_state"). The printer state can be obtained using the above "Fetch Status" API.

Request:

Only one opt-value could be set per request.

```
Python
POST /device/printer_sn/opt
{
    "opt" : "stop" | "pause" | "resume" | "bed_clean"
}
```

Parameters:

- "stop": Stop the current print. This option should be used when the "gcode_state" is "RUNNING" or "PAUSED".
- "pause": Pauses the current print job. This option should be used when the "gcode_state" is "RUNNING".
- "resume": Resumes the current print job. This option should be used when the "gcode_state" is "PAUSED".
- "bed_clean": Confirms that the build plate is ready for a new job. Please ensure that this option is only used when the "gcode_state" is "FINISHED" or "FAILED".

Response:

- HTTP Status Code 200: Success. This indicates that the command has been successfully sent to the printer and the printer has responded with an "OK". Due to the periodic update of device information, the "gcode_state" retrieved from the "fetch Status" operation may change after 10 seconds.
- Any other HTTP status code value: Failure.

4.5 Setting Bed/Nozzle Temperature

To set the temperature for a printer using the BulkOpt API, you can send a POST request to the /device/bulkopt endpoint. The requested body should include the following parameters:

4.5.1 Start BulkOpt

```
Python
Request
POST /device/bulkopt
{
    "dev_ids": ["0XX00A252000006"],
    "opt": "nozzle" | "plate",
    "opt_args": "{ \"degree\": 80 }"
}
```

- "dev_ids": An array of printer serial numbers (SN) that need to be operated on.

- "opt": The operation type. "nozzle" means setting the nozzle temperature, and "plate" means setting the heat bed temperature.
- opt_args: A string in JSON format specifying the temperature setting. For example, "degree": 80 sets the temperature to 80 degrees.

Response:

```
Python
Response
{
  "message": "",
  "code": 0,
  "bulk_id": 1750167002827
}
```

- HTTP Status Code 200 and code is 0: Success, the server will start to send request to printers.
- "bulk_id": this is an ID for this bulk option, which could be used to query printer response result for this operation.

4.5.2 Get BulkOpt Result

```
JSON
Request
POST /device/bulkopt/{bulk_id}
```

```
JSON
Response
{
  "last": true | false,
  "result": [{"dev_id": "01S00A292500496", "message": "", "code": 0}]
}
```

- Https Status Code 200: Please check the "last" and "result" fields to get the printer response.
- Https Status Code 404 with "code" set is 1017: The specified bulk operation could not be found. This may be due to an invalid bulk_id, or the operations for this bulk_id have been completed and the bulk_id has been cleaned up.
- "last": A Boolean value indicating whether the BulkOpt operation has been completed. true means the request has been executed and finished.
- "result": An array where each item provides the printer ID and the operation result.

- `dev_id`: The serial number of the printer.
- `message`: An optional message providing additional information.
- `code`: The result code. 0 means the printer responded successfully. Any other value indicates failure.

4.6 Start Print

4.6.1 3mf File Format

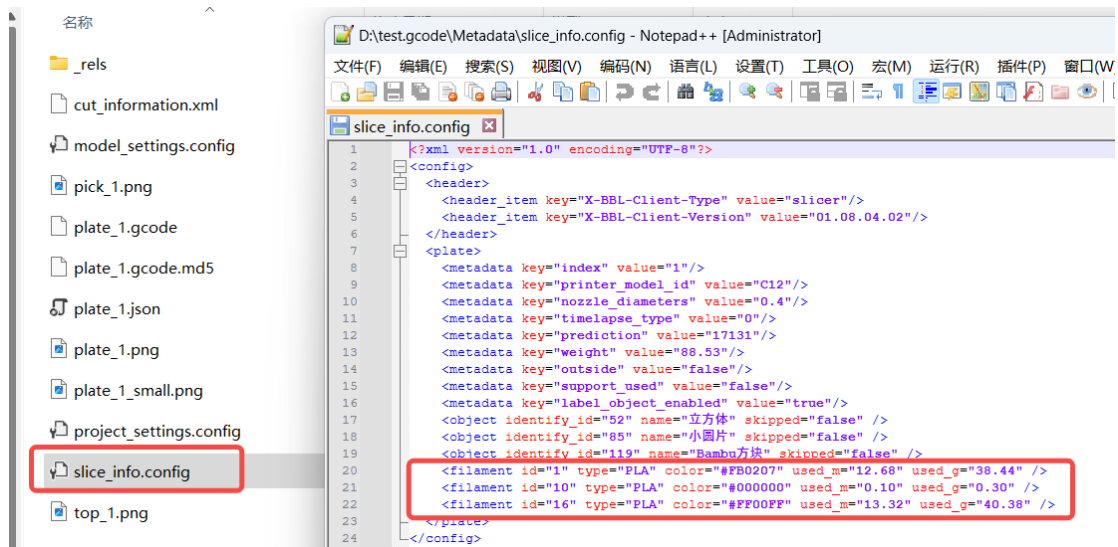
The 3MF file is a compressed ZIP archive. Two types of files need to be parsed for parameter settings of starting a print job.

```
D:\test.gcode\Metadata
2025/06/17 15:34 <DIR> .
2025/06/17 15:34 <DIR> ..
2024/02/05 11:53      299 cut_information.xml
2024/02/05 11:53    2,689 model_settings.config
2024/02/05 11:53     1,509 pick_1.png
2024/02/05 11:53    1,906,346 plate_1.gcode
2024/02/05 11:53        32 plate_1.gcode.md5
2024/02/05 11:53     861 plate_1.json
2024/02/05 11:53    6,053 plate_1.png
2024/02/05 11:53    2,880 plate_1_small.png
2024/02/05 11:53    55,790 project_settings.config
2024/02/05 11:53     1,093 slice_info.config
2024/02/05 11:53    2,459 top_1.png
2025/06/17 15:30 <DIR> _rels
11 个文件      1,980,011 字节
```

- **G-code Files:** Contain print instructions. The actual path (like "Metadata/plate_1.gcode") needs to be specified to start a print.
- **Slice Information Configuration File:** Contains filament information. The actual spool positions for these filaments need to be specified to start a print.

4.6.2 Filament Mapping

The "slice_info.config" file describes which filament will be used for the 3MF file. Each filament has an ID. Filament mapping is used to tell the printer which spool position should be assigned to each filament ID.



Common Format:

The filament mapping is described in the following format:

JSON

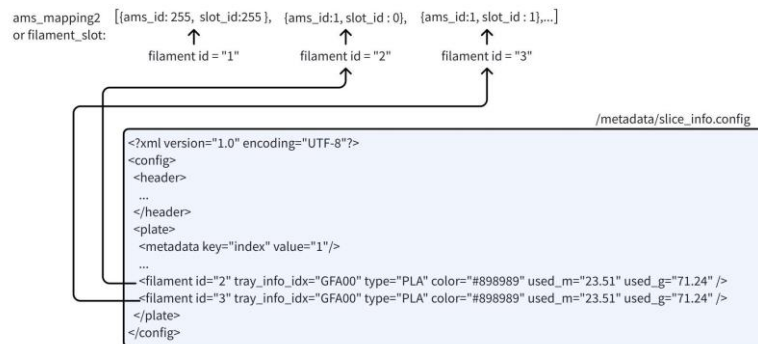
```
[
  { "ams_id": <value>, "slot_id": <value> }, //filament id = "1"
  { "ams_id": <value>, "slot_id": <value> }, //filament id = "2"
  ...
]
```

- Array Items: Each item in the array corresponds to a filament ID, starting from filament 1 up to a maximum of filament 16.
- Complete Mapping: The filament mapping must contain descriptions for all filament IDs from 1 to the maximum filament ID used (which may be less than 16). If some filament IDs are not specified in the config file, they should be set to a default value `{"ams_id": 255, "slot_id": 255}`.
- ams_id and slot_id:
 - ams_id: Identifies the AMS device or external Spool.
 - slot_id: Identifies the specific slot on the AMS device where the filament spool is located.

ams_id	slot_id	Spool Posisson	Name
255	255	Not used	N/A
255	0	External Spool	Ext (X1/A1/P1 series)
254	0	External Spool	Ext (reserved)

128~135 5	0	AMS HT	HT-A (ams_id: 128) ~ HT-H (amd_id: 135)
0	0~3	AMS, AMS Lite or AMS Pro	A1, A2, A3, A4
1	0~3		B1, B2, B3, B4
2	0~3		C1, C2, C3, C4
3	0~3		D1, D2, D3, D4

- Mapping example:



4.6.3 Single-Call API: Upload File & Trigger Print

This API is not the same as launching a queue task from the client UI. It transfers the 3MF file to the server, triggers the selected printer to download the file, and then initiates the printing process. The server will automatically delete the 3MF file after 4 hours to eliminate the need for manual file cleanup operations.

Request:

```
JSON
print_cmd
{
  "dev_id": printer_sn,
  "task_name": "test123",
  "plate": "Metadata/plate_1.gcode",
  "print_option": {
    "auto_bed_leveling": True,
    "flow_dynamic_calibration": True,
    "timelapse": False
  },
  "filament_slot": [{"ams_id": 255, "slot_id": 0}]
}
```

```
}
```

- "print_cmd": The print job parameters. This JSON needs to be serialized to a string and sent as form-data with the name "print_cmd".
- "dev_id": The serial number of the printer.
- "task_name": The job name which will be displayed on the printer side. The total length should be less than 128 bytes.
- "plate": The path from which the printer will abstract the G-code file from the 3MF file. Please reference "6.6.1 3MF File Format".
- filament_slot: This parameter is a reference to "6.6.2 Filament Mapping".

HTTP

PUT/device/printer_sn/print

--boundary_string

Content-Disposition: form-data; name="print_cmd"

1

--boundary_string

Content-Disposition: form-data; name="file"; filename="test.3mf"

Content-Type: application/octet-stream

Response:

JSON

```
{
  "code":0,
  "message":"error description"
}
```

- Https Status Code 200 with "code" set is 0: Success.
- "code" is 1051 and the message is "device busy": The printer is currently printing, upgrading, offline, or requires confirmation of "bed_clean".
- "code" is 1052 and the message is "device internal error": For P/A series printers, this indicates that there is no SD card inserted in the printer.
- "code" is 1053 and the message is "invalid filament setting":The filament mapping is invalid. This could be due to several reasons:
 - There is no filament in the selected AMS slot.
 - The selected AMS does not exist.
 - The ams_id or slot_id is invalid.
- "code" is 1005 and the message is "mqtt request failed": The command sending

to the printer failed. This may be caused by Wi-Fi interference.

- "code" is 1006 and the message is "unbound device": The printer has not been added to the server.
- "code" set is 11: One or more parameters are either not set or are outside the valid range.

4.6.4 Initiate Print Task

Request:

```
HTTP
POST /task
{
  "task_name" : "Cube.gcode.3mf",
  "f3mf_id": "2",
  "device_pool": [
    "01S09A2C0500292"
  ],
  "queue_model_cnt": 1,
  "print_option" : {
    "auto_bed_leveling" : true,
    "flow_dynamic_calibration" : true,
    "timelapse" : true
  },
  "task_print_model": 1,

  "ams_mapping2": [{"ams_id": 0, "slot_id": 2}, {"ams_id": 0, "slot_id": 0},
  {"ams_id": 0, "slot_id": 3}]
}
```

Parameters:

- "task_name": You can specify a task name. We recommend using the name of the 3mf file.",
- "f3mf_id": The file ID obtained from the file upload response.
- "device_pool": it is printer sn array to indicate which printers will start printing
- "user_ams": Indicates the filament spool position, whether it is an external spool or AMS.
- "queue_model_cnt": this should be equal to the number of printers in device_pool.
- "print_option": The 'flow_dynamic_calibration' option is not supported by the P1 series. If set to true, it will not take effect.
- "task_print_model": 1 means the task will be performed immediately after

creation. (0 is task queue mode not support for local server).

- "ams_mapping2": This parameter is a reference to "6.6.2 Filament Mapping".

Response:

```
JSON
{
  "code":0,
  "message":"error description"
}
```

- Https Status Code 200: Success.
- Https Status Code is 500 with "code" set is 1: This typically indicates an error caused by an invalid "f3mf_id" or device_id.